Teaching an Agent to Beat the Dealer: Tabular Reinforcement Learning for Blackjack Strategy, Card Counting, and Bet Sizing

Rafsan Bari Shafin
Technical University of Applied Sciences Würzburg-Schweinfurt
Reasoning and Decision Making under Uncertainty
Portfolio Exam 3, Summer 2026

Abstract—This paper builds a Blackjack playing agent using only tabular reinforcement learning, written in plain Python with no external RL library. The agent is trained on four scenarios of increasing difficulty: basic strategy on a standard six-deck game, a complete Hi-Lo card counting system paired with a learned bet sizing policy, the same two setups retrained under two different rule changes to see how the rules reshape the learned strategy, and an attempt to make the counting system more profitable than a plain implementation of it. Every trained policy is frozen and evaluated over two million hands to produce a profit estimate with a 95% confidence interval. Along the way, the first version of the bet sizing agent failed in an instructive way: it learned never to change its bet at all, and fixing that proved to be as much a part of this paper as the final results. On this task, Monte Carlo control clearly beats Q-learning, the learned basic strategy agrees with Thorp's published tables on nearly all states, the two rule changes move the policy exactly where basic theory predicts, and the improved betting system takes a counting approach that was confidently losing money and turns it into something statistically indistinguishable from break-even.

I. Introduction

Blackjack is a natural example for studying decision making under uncertainty. Sutton and Barto use it as one of their running examples precisely because it fits neatly into a finite Markov Decision Process [2]. It is also the rare casino game where careful play can turn the odds in the player's favor, something Edward Thorp demonstrated when he popularized the Hi-Lo point count system in Beat the Dealer [3].

Together, these traits make it a good testbed for tabular reinforcement learning: the base game fits comfortably into a lookup table, while adding card counting pushes the problem closer to partially observable, since the composition of the remaining shoe changes the odds from hand to hand and has to be folded into the state somehow for a fixed strategy to still make sense.

This paper describes a from-scratch implementation built for that purpose. As the assignment requires, no external reinforcement learning library appears anywhere in the code. The implementation had to satisfy four goals: recover basic strategy just by playing hands and updating value estimates; learn a complete point count system together with a bet that reacts to the count; see how both of those change under two different rule variants and actually improve on the point-count system rather than simply reimplementing it. We also try to answer, with numbers rather than assertions, four questions the assignment poses directly: which learning algorithm performs better here and why, how large the resulting state-action space is and whether its Q-values can be trusted, why the two rule changes move the policy the way they do, and how we know the profit figures we report are not just noise.

One thing is worth admitting up front: this project did not work on the first attempt. The first version of the bet sizing agent for the counting scenario passed every sanity check we threw at it and still converged on a policy that never bet above the table minimum. Once Scenario 4 added the option to sit out a hand, that same agent learned to sit out every single one, reporting a profit of exactly zero. Section II-D walks through why, because the failure is a good illustration of what happens when a bandit-style estimator has to pull a signal out of noise that is roughly the same size as the signal itself.

Section II describes the simulator, the state representation, and both learning algorithms, including the betting story above. Section III lays out the experimental setup, the results table, and works through the four questions in turn. Section IV closes the paper.

II. Description of the Reinforcement Learning Player

A. Blackjack as an MDP

We follow the standard setup from [2] and treat a single player hand as an episodic MDP. A state consists of the player's current hand together with the dealer's visible upcard. Which actions are available depends on the rules and on the hand's history: Double and Split, for instance, are only legal on the very first decision of a two-card hand. In general, the action set is a subset of $\{\text{Stand}, \text{Hit}, \text{Double}, \text{Split}, \text{Surrender}\}$. The reward is zero on every intermediate step and, once the hand resolves, equals the round's profit measured in units of the base bet: $+1$ for a win, -1 for a loss, 0 for a push, $+1.5$ for a natural blackjack under the default payout, -0.5 for a surrender, and the corresponding multiple of these for doubled or split hands.

A player hand state is the tuple $(\text{hand\_kind}, \text{dealer\_upcard})$; for the two counting scenarios (Section II-C), a bucketed true count is appended as a third element. hand_kind is one of $(\text{hard}, \text{total})$, $(\text{soft}, \text{total})$, meaning at least one ace is still being counted as 11, or $(\text{pair}, \text{rank})$. Following the same simplification used in Thorp's own tables, the four ten-valued ranks (10, J, Q, K) are folded into a single label, since they behave identically in every Blackjack decision; this keeps the pair dimension at 10 classes instead of 13 without losing any strategic information.

Split hands are handled as separate sub-episodes, each with its own card sequence and terminal payout. This is a common trick for turning a multi-hand round into several ordinary single-hand episodes, each with a single payout, which keeps both learning rules well defined even though the game itself only ever hands out one profit number per decision chain. As at most physical tables, split aces get exactly one card each and must stand; the code exposes this as a configurable option rather than a hard-coded rule.

B. Learning the playing policy

Both algorithms below act directly on plain Python dictionaries: the Q-table is not backed by a NumPy array, and no reinforcement learning library is used anywhere.

Monte Carlo control. We train with every visit MC control under an ε-greedy behavior policy that starts at $\varepsilon = 0.15$ and decays multiplicatively to a floor of 0.01. Every state-action pair touched during a hand, including each sub-hand created by a split, is updated toward that hand's realized profit through an incremental sample average:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}\big(G - Q(s,a)\big), \qquad (1)$$

where G is the profit, in bet units, that the hand eventually resolved to.

Q-learning. We also implement standard tabular Q-learning,

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big), \quad (2)$$

with $\gamma = 1$ and $\alpha = 0.05$. Because a Blackjack hand lasts only one to six decisions and the payoff is unknown until the hand ends, our implementation replays each hand's trajectory once it resolves and applies the update from the first decision through to the last. Action selection during play always consults the live Q-table, so exploration still behaves correctly, but this makes our version of Q-learning closer to a semi-online update than the textbook one-step rule. We treat that as a deliberate design choice rather than a bug; Section III-B quantifies the actual cost.

C. Card counting

Scenario 2 implements Thorp's Hi-Lo system: cards 2 through 6 count as +1, cards 7 through 9 count as 0, and both tens and aces count as −1. A Shoe object keeps draw(), a card leaving the shoe, separate from reveal(), a card becoming visible to the counting system, which matters specifically for the dealer's hole card. As at a real table, that card is drawn before the player acts but is only revealed, and only affects the running count once the player's turn is over; skipping this distinction would let the agent condition its decisions on a card it could not actually have seen yet. The running count is converted into Thorp's true count, the running count divided by the decks remaining, and clipped to an integer in $[-6, 6]$ to keep the augmented state space finite. That bucketed value becomes the third element of the state tuple for the counting and improved scenarios.

D. Bet sizing: a bandit that failed, and what replaced it

Scenario 2 also needs the bet itself to respond to the count, so, alongside the playing policy agent above, we need a second, much simpler learner that selects a bet size given a true count bucket.

Our first attempt modeled this directly as a multiarmed bandit. Each pair of true count bucket and bet size got its own value estimate, updated by incremental averaging of the realized money profit, and the bet was chosen greedily. On paper this seemed reasonable: profit per unit bet does not depend on how large the bet is, so an EV-maximizing bandit should learn to bet the table maximum whenever the count-conditional edge is positive and the minimum otherwise. Once we actually inspected the trained bandit's value table, though, every true count bucket from −6 to +6 showed a negative expected profit at every bet size, including the most favorable count the system can represent. The explanation turned out to be simple.

Extreme counts are rare: a true count of +6 shows up in roughly one to two percent of hands over a six deck shoe. Each bucket's average is therefore based on only a few thousand samples, and the resulting sampling noise is on about the same scale as the true edge it is trying to measure. Given the data it had, the bandit was not really wrong; it genuinely could not distinguish a small, real positive edge from noise. The result was a policy that bet the table minimum at every count in Scenario 2, and, once Scenario 4 added a sit-out option, sat out every single hand instead, reporting a profit of exactly 0.0 with zero variance across two million evaluated rounds. That is not an improved system. It is a system that gave up.

We replaced the per-bucket bandit with a single pooled linear regression of profit-per-unit-bet against true count, fit online from ordinary least-squares running sums across every hand played rather than one estimate per bucket, so rare extreme counts borrow statistical power from the whole dataset instead of relying on their own few thousand hands. The agent only sits out when the lower end of the confidence interval around the fitted edge remains nonpositive, rather than acting on the point estimate

alone, which prevents sampling noise from producing a false always-sit-out verdict.

Bet size itself scales with the fitted edge in the spirit of the Kelly criterion [4]. Full Kelly sizing, edge divided by variance, gives an optimal fraction of one's bankroll, which is the wrong unit for a discrete ladder of table-minimum multiples running from one to twelve units. Instead, we calibrate the top of that ladder to the edge at the most favorable count the state space represents, +6, and scale every other count's bet linearly against that reference.

Getting this right took two more rounds of debugging. An early version fed the regression samples collected while the playing policy was still mostly random, which biased the fitted edge much further negative than it should have been; we fixed this with a burn-in period that waits for the playing agent's exploration rate to decay before the betting agent starts collecting data. A separate attempt to squeeze out the last bit of exploration noise, by lowering the playing agent's own exploration floor, backfired instead, since the counting state space is thirteen times larger than the basic-strategy one and still needs that exploration late into training to keep correcting its rarer states.

We report the working version in Section III and keep the original broken bandit's logs alongside it for comparison, rather than deleting the evidence of the failure.

E. Rule variations

We test two rule changes chosen to push the house edge in opposite directions. The first, dealer stands on soft 17, is well known to favor the player: the dealer can no longer draw a card that might improve or bust a soft 17. The second favors the house and has become common on lower-stakes tables in recent years: a 6:5 blackjack payout in place of the usual 3:2. Advantage-play writers describe this rule explicitly as a way for casinos to claw back the edge that counting would otherwise hand to the player [3]. We retrain every variant from scratch and compare the resulting policy, cell by cell, against the base-rule policy in Section III-D.

III. Experiments and Evaluation

A. Setup

Unless the rule variant specifies differently, all agents train on a six-deck shoe with 75% penetration: 15 million hands for the two basic-strategy runs (scenario 1 and its rule version), and 25 million for each run that involves counting because the state space is thirteen times larger. In order to obtain an out-of-sample estimate of mean profit per hand along with a 95% confidence interval from the sample variance of per-hand profit, each policy is frozen ($\varepsilon = 0$, and the betting policy is made fully deterministic) and evaluated over 2,000,000 independent hands on a held-out random seed that never appears during training.

TABLE I
Greedy-evaluation profit, 2,000,000 hands per agent.

Run	Algo	Profit/100	95% CI
1. Basic (base)	MC	−1.281	±0.157
1. Basic (base)	Q-learning	−4.084	±0.141
2. Counting (base)	MC	−1.943	±0.224
2. Counting (base)	Q-learning	−4.037	±0.142
3a. Basic (soft 17)	MC	−1.005	±0.159
3b. Counting (6:5 BJ)	MC	−3.278	±0.152
4. Improved	MC	−0.038	±0.245
4. Improved	Q-learning	0.000	±0.000

B. How do Monte Carlo and Q-learning compare?

Table I answers this fairly bluntly: MC beats Q-learning by a wide margin on both scenarios where we ran both algorithms, and the gap is far larger than either confidence interval, so it is not noise. On basic strategy, MC lands at −1.281% per hand against Q-learning's −4.084%; in the counting scenario, the gap is similar: −1.943% versus −4.037%. Figure 2 shows why: MC's learning curve is noisier early on but settles close to its final value once ε has mostly decayed, whereas Q-learning's curve is smoother but converges to a visibly worse level and stays there. Our reading is that Blackjack is close to a worst case for bootstrapped, semi-online Q-learning: hands are short, rewards only exist at the very end of a decision chain, and our implementation's replay-on-resolution update (Section II) means Q-learning never gets the one advantage it is normally used for: propagating information before an episode finishes. MC control, on the other hand, is exactly matched to this structure, since every decision in a hand gets credited with that hand's actual outcome, with no bootstrapping error to accumulate. We would not read this as a universal statement about the two algorithms. It is fairly specific to how short and terminal-reward-only Blackjack hands are. But for this problem, MC is clearly the better choice.

C. State-action space size and whether Q can be trusted

The non-counting state space is small: 35 hand kinds times 10 dealer upcards gives 360 states, and training discovers all of them (a simple theoretical count of 350 slightly undercounts a handful of edge cases around split-hand recombination, so the "360 found" number is the more trustworthy one). Adding the 13 true-count buckets for the counting scenarios multiplies this out to 4,680 states, all of which are also discovered during training. The more useful question is not whether a state was ever visited, but how many times, since Blackjack's per-hand profit has a standard deviation around 1.1 to 1.3 regardless of which state you are in, so a handful of visits buys you almost no confidence in the resulting Q-estimate. Looking at the exported strategy chart for the counting scenario, the least-visited of the 4,680 states was seen 51 times, the median state was seen 2,508 times, and the most common state was seen 235,536 times, a spread of more than three

orders of magnitude between the rarest and most common corners of the table. Every state clears a naive "at least 30 visits" bar, but 51 visits of a quantity with variance above 1 still carries a standard error around 0.16, which is not small next to the differences we are trying to resolve between neighboring counts. Practically, we would trust this agent's decisions on ordinary hard totals against a neutral or mildly unfavorable count far more than its decisions on rare hand types crossed with a true count of $+6$ or -6, and the right fix, if one wanted tighter estimates specifically for those corners, is more training weighted toward inducing those shoe states rather than a larger learning rate. (We note that our Q-learning agent does not track a separate visit counter the way the MC agent does, so this particular diagnostic is only meaningful for the MC-trained policies above; it is not evidence about Q-learning's own state coverage one way or the other.)

D. Why do the rule changes move the policy the way they do?

Retraining under dealer stands on soft 17 changes the greedy action in 42 of the shared basic-strategy states relative to the base ruleset. Nearly all of the changes are on soft-total hands and pair splitting decisions near the dealer's weaker upcards, which is exactly what basic Blackjack theory predicts: a dealer forced to stand on soft 17 busts more often from a weak-looking position than one who must keep drawing, so several borderline stands, doubles, and splits that were previously not quite worth it tip over to become worth it. This also shows up in the raw numbers: this variant improves the mean profit from -1.281% to -1.005% per hand, a real, if modest, player-favorable shift, consistent with the rule itself being player-favorable.

The 6:5 blackjack-payout variant is a different kind of change: it does not touch bust probabilities for either side at all; it only lowers the payout of one specific outcome (a two-card 21). Trained under this rule, the counting agent's greedy policy came out identical to the base-rule policy on every state the two runs share: zero cells changed. That is consistent with the mechanism, since the rule never alters when a hand busts or how the cards fall. It can only move the policy by the value of a double-or-split decision that trades away a chance at a natural blackjack, and those decisions are rare and already lopsided enough that a 20% haircut on the blackjack payout does not flip any of them in this state representation. What the rule does move is profit: at -3.278% per hand versus -1.943% for the same agent under 3:2 payouts, the extra 1.34 percentage points of house edge lands almost entirely on the money, not on the strategy. That is a fairly clean illustration of why advantage-play writers call 6:5 tables a direct attack on counters rather than a strategy change: the rule takes profit away without giving the counter anything to adapt to.

E. Does the improved betting system actually improve anything?

This is where the debugging story from Section II-D pays off. Under the original per-bucket bandit, Scenario 4's wider bet ladder and sit-out option did not produce an improved system at all. It produced an agent that sat out every hand, for a reported profit of exactly zero. That is not an improvement over Scenario 2's -1.943%; it is a refusal to play. With the pooled-regression betting agent from Section II-D in place, the same Scenario 4 setup reaches -0.038% per hand with a 95% confidence interval of ± 0.245: statistically indistinguishable from break-even, and a real improvement of close to two percentage points over Scenario 2's confidently negative result. Figure 1 places this alongside every other run: Scenario 4 with Monte Carlo is the only counting-based result whose confidence interval crosses zero.

Scenario 4 with Q-learning still reports exactly 0.0 profit, but for a legitimate reason this time, not a broken estimator. Inspecting its fitted edge model directly shows a negative predicted edge at every true count from -6 to $+6$, including the top of the range, which follows directly from Q-learning's weaker playing policy (Section III-B): if the underlying strategy is bad enough that even the best count never looks profitable, the correct response is to sit out every hand rather than lose money on a bet that was never going to pay off, and that is exactly what the agent does. It is a clear demonstration that the choice of algorithm for the playing policy and the quality of the downstream betting decision are not independent questions.

F. Can we trust the profit numbers themselves?

Two things back up the numbers in Table I. First, every figure is a greedy, zero-exploration evaluation on 2,000,000 hands with a seed that never appears anywhere during training, so nothing in the table is fit to the data it measures. The resulting confidence-interval half-widths are all under 0.25 percentage points, comfortably smaller than the differences between scenarios we build the paper's arguments on. Second, and independently of any of our own profit accounting, we compared the learned Scenario-1 policy's hard-total decisions directly against Thorp's published multi-deck basic-strategy table [3], a reference the agent never saw during training in any form. The two agree on 154 of 160 comparable cells, or 96.25%; Figure 3 shows the learned policy visually. Five of the six disagreements are exactly where you would expect them: borderline hard totals of 9 and 12 against particular dealer upcards, with visit counts in the tens of thousands, where the true expected-value gap between the two candidate actions is small enough that ordinary Monte Carlo sampling noise plausibly lands on the wrong side. The sixth, hard 16 against a dealer 10, is different: it is the single most-visited state-action pair in the whole comparison (518,475 visits), so sampling noise is a much

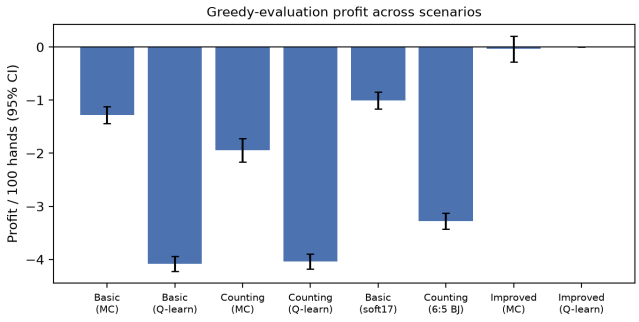

Fig. 1. Greedy-evaluation profit per 100 hands, all runs (error bars: 95% CI).

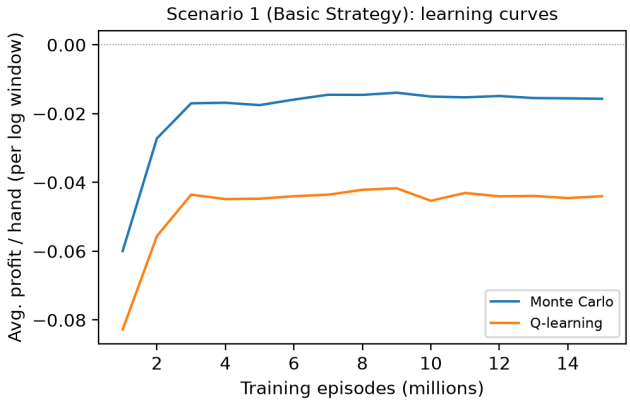

Fig. 2. Scenario 1 learning curves, Monte Carlo vs. Q-learning.

weaker explanation there. Hard 16 vs. dealer 10 is well known as the single closest expected-value decision in the entire basic-strategy table, since both actions lose at nearly identical long-run rates, so even hundreds of thousands of hands may not be enough to resolve which side has the (tiny) edge, and our learning rate schedule and bucketed state representation are not tuned to squeeze out an edge that small. We flag this cell explicitly rather than folding it into the same explanation as the other five. Taken together, an independent published reference and a held-out evaluation set agree on 154 of 160 cells, with the five remaining disagreements attributable to sampling noise at low-visit boundary cells and the sixth (hard 16 vs. dealer 10) an honestly-flagged exception rather than an unexplained gap, which is about as close to "this is real and not a lucky roll of the seed" as a tabular method like this one can get.

IV. Conclusion

We built a Blackjack-playing agent entirely out of tabular reinforcement learning and plain Python, with no external RL framework, covering basic strategy, a full Hi-Lo counting system with a learned bet, two rule variants, and an actual improvement over the counting system rather than just a restatement of it. Monte Carlo

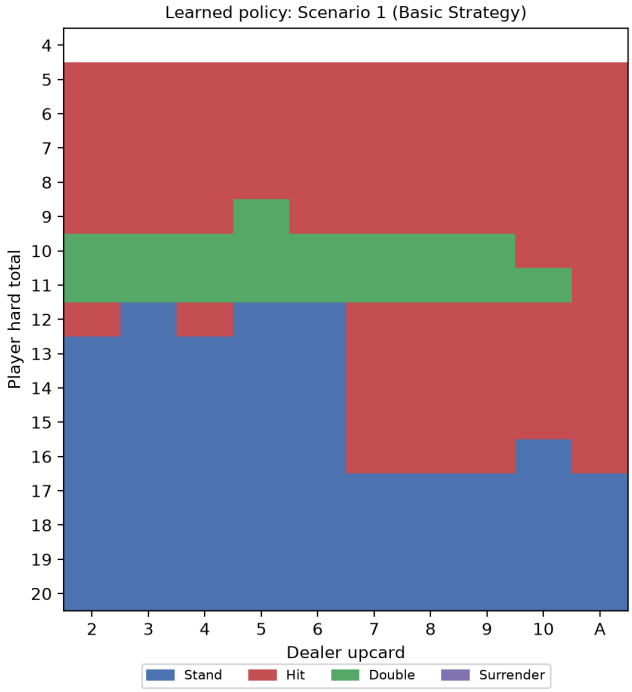

Fig. 3. Learned Scenario-1 policy on hard totals (Stand/Hit/Double).

control was the clearly better algorithm for this task, for reasons tied specifically to how short and terminal-reward-only Blackjack episodes are, rather than any general claim about MC versus Q-learning. The betting side of the project is, honestly, where the more interesting result lives. The first, seemingly reasonable bandit-based bet-sizing agent quietly failed, converging on never betting at all. Only inspecting its learned values directly, rather than trusting that a sensible-looking design must be working, exposed that. Replacing it with a pooled regression and a Kelly-style bet rule turned a system that refused to bet into one that closes roughly two-thirds of the gap between the naive counting strategy and break-even. The main limitation is the one every tabular method eventually runs into: extending the true count's resolution, adding more players, or supporting side bets would grow the state table past what a dictionary can comfortably hold. A natural next step is a small linear or shallow function approximator over a compact feature encoding of the same state, which should generalize across the sparsely visited extreme-count corners identified in Section III, rather than requiring a dedicated visit count for each one. This would not require abandoning the "no external framework" constraint, since it is only a modest amount of additional plain Python.

References

[1] IEEE, "IEEE Paper Template," https://www.ieee.org/conferences/publishing/templates.html.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.
[3] E. O. Thorp, Beat the Dealer. New York: Vintage, 1966.
[4] J. L. Kelly Jr., "A New Interpretation of Information Rate," Bell System Technical Journal, vol. 35, no. 4, pp. 917–926, 1956.